TalknShop: AI-Powered Conversational Shopping Assistant

Sameer Sah
Dept. of Computer Engineering
San Jose State University
San Jose, CA, USA
sameer.sameer@sjsu.edu

Puneet Bajaj
Dept. of Computer Engineering
San Jose State University
San Jose, CA, USA
puneet.bajaj@sjsu.edu

Rutuja Nemane
Dept. of Computer Engineering
San Jose State University
San Jose, CA, USA
rutujabhagwat.nemane@sjsu.edu

Kalpesh Anil Patil
Dept. of Computer Engineering
San Jose State University
San Jose, CA, USA
kalpeshanil.patil@sjsu.edu

Abstract

The rapid growth of e-commerce has created a fragmented marketplace landscape where consumers must navigate multiple platforms to find products and sellers must redundantly list items across several marketplaces. This paper presents TalknShop, a conversational AI platform that unifies product search and listing across multiple marketplaces through natural language chat. The system employs a microservices architecture coordinated by an orchestrator service built on LangGraph state machines and powered by AWS Bedrock (Claude 3). Buyers can search across eBay, Amazon, Walmart, and Best Buy simultaneously, while sellers can cross-post listings to eBay, Craigslist, Facebook Marketplace, and Poshmark—all via a single conversation. The platform uses WebSocket-based real-time communication, asynchronous SQS-based job processing, and DynamoDB for state persistence. We describe the system architecture, conversation flow design, marketplace integration strategy, and propose an evaluation framework with visualizations for measuring user efficiency, satisfaction, and purchase outcomes.

Index Terms— *conversational commerce, AI shopping assistant, microservices, LangGraph, large language models, multi-marketplace integration, natural language processing*

1 Introduction

The global e-commerce market continues to grow, yet the consumer experience remains fragmented. Buyers seeking a product must individually visit Amazon, eBay, Walmart, and other platforms, each with distinct search interfaces and result formats. Sellers face a mirror-image problem: listing a single product on multiple marketplaces requires navigating different interfaces, formats, and posting workflows. A consumer comparing laptop prices, for example, must open multiple browser tabs, reenter search criteria on each site, and mentally reconcile results that use different product taxonomies and pricing structures.

The conversational commerce market was valued at approximately $8.8 billion in 2025 and is projected to reach $32.6 billion by 2035 at a 14.8% CAGR [1]. Research shows shoppers complete purchases 47% faster with AI assistance, and over 54% of consumers report increasingly conversational search habits [2]. The U.S. AI shopping assistant market alone is expected to grow significantly as retailers invest in intelligent interfaces [11] [12]. Despite these trends, existing solutions—Amazon Rufus, ChatGPT Shopping, Perplexity Buy with Pro—remain confined to single-marketplace ecosystems or lack deep integration for both buying and selling [3].

This paper presents TalknShop, a conversational AI platform that addresses this gap. The primary contributions are:

1. A unified conversational interface for multi-marketplace buying and selling.
2. A microservices architecture separating synchronous buyer search from asynchronous seller listing operations.
3. A LangGraph-based state machine for complex multi-turn conversation management.
4. A marketplace adapter pattern abstracting heterogeneous marketplace APIs.
5. A proposed evaluation framework for conversational commerce systems.

2 Related Work

2.1 Conversational Commerce

The term "conversational commerce" was introduced by Messina in 2016 [4]. Rese et al. demonstrated that perceived anthropomorphism is the most significant factor in user attitudes toward digital shopping assistants [5]. Major platforms have since invested heavily: Amazon launched Rufus in 2024 [3], OpenAI introduced ChatGPT Shopping Research and Instant Checkout in 2025 [6] [7], and Perplexity AI launched research-focused shopping with source transparency [6]. Walmart's 2025 Retail Rewired Report further highlights that retailers see conversational interfaces as a key differentiator, with

72% planning AI-driven shopping experiences within two years [14]. McKinsey's 2025 global survey corroborates this, finding that AI adoption in retail has doubled since 2023, with personalized recommendation and conversational search as the leading use cases [15]. However, none of these solutions address the cross-marketplace integration challenge that TalknShop targets.

2.2 Agent Orchestration and Microservices

LangGraph provides a low-level framework for building stateful, multi-actor applications using graph-based state machines with durable execution, human-in-the-loop, and persistent checkpointing [8]. Microservices architecture has become standard for modern e-commerce, enabling independent deployment and failure isolation [9] [13]. The CQRS (Command Query Responsibility Segregation) pattern, separating read and write operations, is widely adopted in event-driven commerce systems where read and write workloads have fundamentally different performance and scaling characteristics [10]. TalknShop applies this pattern at the service level: the Catalog Service handles read-heavy buyer searches synchronously, while the Seller Crosspost Service processes write-heavy listing operations asynchronously via message queues. To our knowledge, TalknShop is the first system to combine conversational AI, multi-marketplace integration, and production-oriented microservices for both buying and selling within a single platform.

3 System Architecture

3.1 Architectural Overview

TalknShop follows a microservices architecture with five services: the Orchestrator Service, Catalog Service, Seller Crosspost Service, Media Service, and client applications (web and iOS). Fig. 2 illustrates the high-level architecture.

3.2 Orchestrator Service

The Orchestrator Service (Port 8000) is the central coordination hub, built with FastAPI and WebSocket. It integrates LangGraph for state machine execution and AWS Bedrock (Claude 3) for natural language understanding and response generation. DynamoDB provides durable state persistence with sub-millisecond read latency for session retrieval. The service contains two LangGraph graphs: a 10-node buyer flow and a 12-node seller flow. Ranking and composition logic runs as a node within the Orchestrator rather than as a separate service, reducing inter-service latency for the buyer flow. WebSocket connections enable real-time streaming of partial results back to clients as marketplace API responses arrive, providing progressive feedback rather than requiring the user to wait for all marketplaces to respond.

3.3 Catalog Service (Buyer Flow)

The Catalog Service (Port 8002) executes parallel searches across eBay, Amazon, Walmart, and Best Buy adapters, aggregates results, and performs multi-factor ranking. It is synchronous (1–3 second response), stateless, and uses Redis caching for frequently queried products.

3.4 Seller Crosspost Service (Seller Flow)

The Seller Crosspost Service (Port 8003) manages asynchronous listing via an SQS-based job queue. Posting requires 8–15 seconds per marketplace due to rate limits and image uploads. The service validates listings, dispatches per-marketplace SQS messages, and tracks job status in DynamoDB with retry logic and partial success handling. Supported marketplaces include eBay, Craigslist, Facebook Marketplace, and Poshmark.

3.5 Media Service and Clients

The Media Service (Port 8001) provides speech-to-text and image attribute extraction via AWS Bedrock. Client applications include a React+TypeScript web app communicating via WebSocket and a planned iOS native app built with Swift.

4 Conversation Flow Design

4.1 Buyer Flow (10-Node Graph)

The buyer flow comprises ten LangGraph nodes: `ParseInput` → `NeedMediaOps` → `MediaProcessing` → `BuildRequirement` → `NeedClarify` → `AskClarifyingQ` (loop) | `SearchMarketplaces` → `RankAndCompose` → `Done`.

[FIGURE PLACEHOLDER]

Buyer flow LangGraph state machine showing 10 nodes with conditional edges for clarification loops and media processing branches.

Figure 1: Buyer flow LangGraph state machine with clarification loop.

The `NeedClarify` node evaluates whether accumulated requirements are sufficient. If ambiguities remain, the graph routes to `AskClarifyingQ`, generating a targeted question. The user's response (type `ANSWER`) resumes the graph from the checkpoint. This loop executes until requirements are sufficient for a productive search.

4.2 Seller Flow (12-Node Graph)

The seller flow extends the buyer pattern with additional nodes for listing validation, marketplace selection confirmation, and asynchronous job tracking with real-time progress updates as each marketplace posting completes. The additional nodes handle image upload to S3, automated description generation from product photos using AWS Bedrock's multimodal capabilities, and

per-marketplace format translation. A confirmation node presents a preview of the listing across all selected marketplaces before dispatching, allowing users to adjust details before committing.

4.3 Session Management

Each conversation is identified by `session_id` and `user_id`. LangGraph state is checkpointed to DynamoDB after each node execution, enabling durable execution that survives service restarts and network disconnections. This checkpointing mechanism is critical for the seller flow, where asynchronous posting operations may span several minutes. If a user closes the app mid-posting, they can reconnect and the system resumes from the last checkpoint, displaying the current status of each marketplace submission.

5 Marketplace Integration

TalknShop employs a marketplace adapter pattern to abstract heterogeneous APIs behind a unified interface. Each adapter translates between internal data models (`RequirementSpec` for buyer, `ListingSpec` for seller) and marketplace-specific formats. Read and write adapters are deliberately separated across services due to fundamentally different operational characteristics. Table 1 summarizes these differences.

Table 1: Buyer Flow vs. Seller Flow Comparison

Aspect	**Buyer Flow**	**Seller Flow**
Service	Catalog Service	Seller Crosspost
Operation	Search (READ)	Post (WRITE)
Execution	Synchronous	Async (SQS)
Response Time	1–3 seconds	30s – 5 min
Graph Nodes	10	12
Data Object	RequirementSpec	ListingSpec
SLA Target	99.9%	95%

5.1 Buyer-Side Adapters

Each buyer adapter implements a common `search(RequirementSpec)` interface that returns a normalized list of product results. The eBay adapter uses the Browse API with aspect filters, while the Amazon adapter translates requirements into the Product Advertising API's keyword and category parameters. The Walmart adapter queries the Open API with department-level filtering, and the Best Buy adapter leverages the Products API with attribute-based refinement. Results from all adapters are merged into a unified schema containing price, condition, shipping cost, seller rating, and marketplace source. The Catalog Service executes these adapter calls in parallel using Python's `asyncio.gather()`, reducing total search latency to the slowest individual marketplace response rather than the sum.

5.2 Seller-Side Adapters

Seller adapters implement a `post(ListingSpec)` interface that translates a canonical listing into marketplace-specific formats. Each marketplace imposes distinct constraints: eBay requires item specifics drawn from a controlled vocabulary, Craigslist expects plain-text descriptions with geographic metadata, Facebook Marketplace mandates category selection from a fixed taxonomy, and Poshmark enforces brand and size attributes for fashion items. The adapters handle these translation requirements transparently, and image resizing is performed by the Media Service before handoff. Partial success handling ensures that a failure on one marketplace does not block others—each posting is tracked independently in DynamoDB with per-marketplace status fields. Rate limiting is managed per adapter: eBay's API permits 5,000 calls per day, while Craigslist imposes stricter throttling that requires exponential backoff. The SQS-based queue absorbs these differences, retrying failed postings up to three times before marking a marketplace as failed.

5.3 Result Ranking and Normalization

When the Catalog Service returns aggregated results from multiple marketplaces, the Orchestrator's `RankAndCompose` node applies a multi-factor scoring function. Each result is scored on price competitiveness (normalized within the result set), seller reputation (mapped to a 0–1 scale across marketplaces), shipping cost and speed, product condition, and return policy availability. The weights are currently fixed but designed to be replaced by a learned preference model in future iterations. The top-ranked results are presented conversationally, with the LLM composing a natural-language summary that highlights trade-offs—for example, noting that a lower-priced eBay listing ships slower than a slightly more expensive Amazon Prime option.

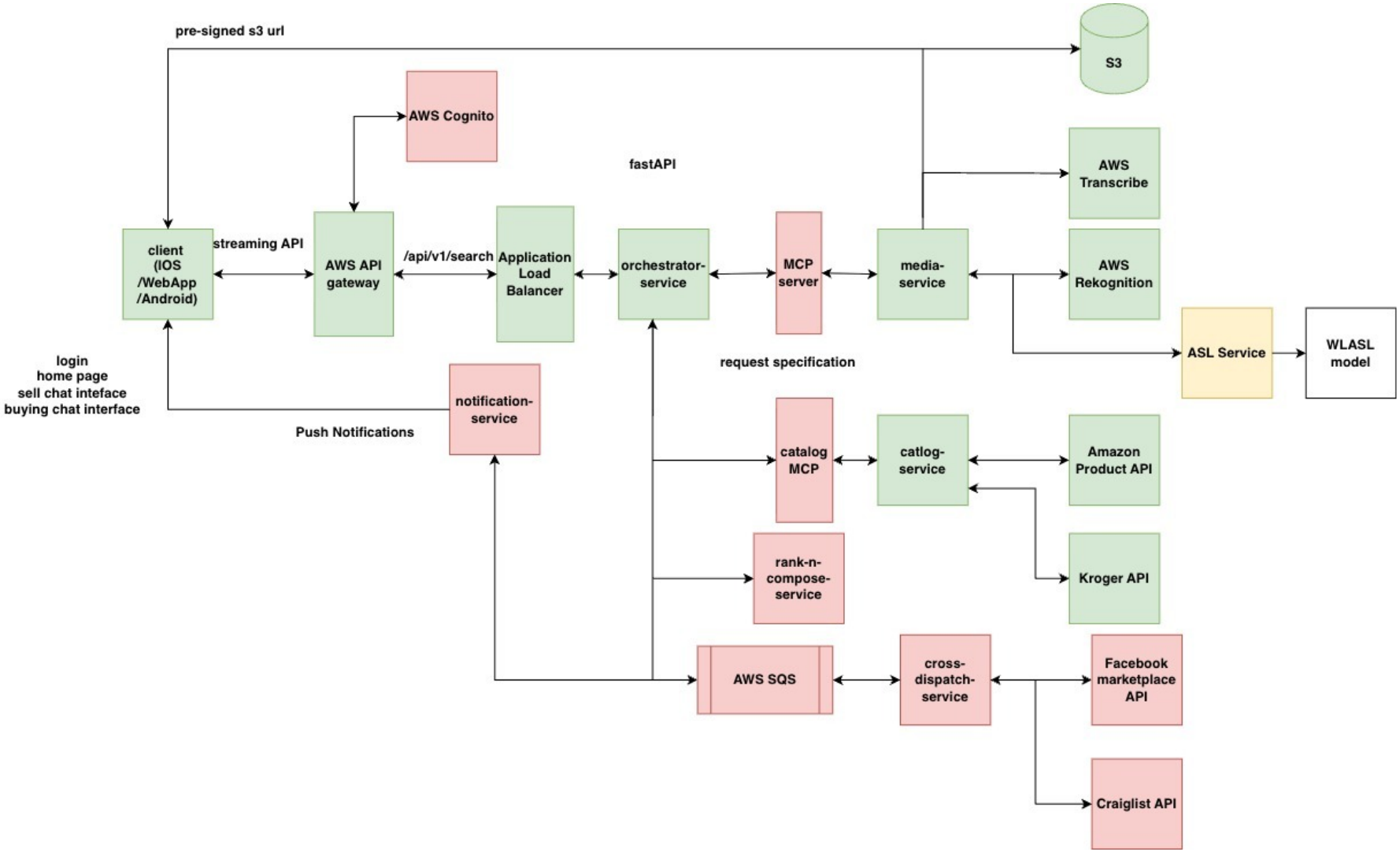

Figure 2: High-level system architecture showing microservices decomposition and marketplace integration.

6 Proposed Evaluation Framework

We propose a between-subjects study comparing TalknShop against traditional marketplace websites across identical shopping tasks. The following visualizations will form the empirical contribution once data is collected.

6.1 Task Completion Time (Fig. 3)

[GRAPH PLACEHOLDER]

Grouped bar chart with error bars.
X-axis: Task type (Simple Search, Multi-Criteria, Cross-Marketplace, Single Listing, Multi-Listing).
Y-axis: Avg. time (seconds).
Bars: TalknShop vs. Traditional.
Hypothesis: Greatest time reduction for cross-marketplace comparison tasks.

Figure 3: Task completion time: TalknShop vs. traditional interfaces.

6.2 Purchase Conversion Funnel (Fig. 4)

[GRAPH PLACEHOLDER]

Side-by-side funnel chart.
Stages: Query → View → Compare → Cart → Purchase.
Two funnels: TalknShop vs. Traditional.
Baseline: AI-assisted 12.3% vs. unassisted 3.1% [1].

Figure 4: Conversion funnel comparing drop-off rates at each stage.

6.3 System Usability Scale Scores (Fig. 5)

[GRAPH PLACEHOLDER]

Box-and-whisker plot.
X-axis: Interface (TalknShop, eBay, Amazon, Walmart).
Y-axis: SUS Score (0–100). Reference line at SUS=68.

Figure 5: System Usability Scale score distribution across interfaces.

6.4 Conversation Turns to Completion (Fig. 6)

[GRAPH PLACEHOLDER]

Violin plot.
X-axis: Number of turns. Distributions for Simple, Complex, and Seller tasks.
Characterizes conversational overhead by task type.

Figure 6: Distribution of conversation turns by task complexity.

6.5 Cross-Marketplace Price Distribution (Fig. 7)

[GRAPH PLACEHOLDER]

Scatter plot with box overlay.
X-axis: Marketplace. Y-axis: Price for identical queries.
Line: TalknShop "best deal" selection.
Demonstrates value of unified cross-marketplace search.

Figure 7: Price variance across marketplaces for identical queries.

6.6 Response Latency Breakdown (Fig. 8)

[GRAPH PLACEHOLDER]

Stacked bar / waterfall chart.
X-axis: Query type. Y-axis: Time (ms).
Segments: WebSocket, LLM inference, API calls, Ranking.
Validates sync/async architecture separation.

Figure 8: Latency breakdown by service component per query type.

6.7 Seller Listing Efficiency (Fig. 9)

[GRAPH PLACEHOLDER]

Line chart with confidence intervals.
X-axis: # Target marketplaces (1–4). Y-axis: Total time (min).
Lines: TalknShop (parallel SQS) vs. Manual (sequential).
Hypothesis: Sublinear vs. linear time growth.

Figure 9: Multi-marketplace listing time: parallel vs. sequential.

6.8 Clarification Rounds vs. Quality (Fig. 10)

[GRAPH PLACEHOLDER]

Dual-axis line chart.
X-axis: Clarification rounds (0–3+).
Y1: Relevance score (0–10). Y2: Task time (sec).
Determines optimal clarification count.

Figure 10: Impact of clarification rounds on search quality and time.

6.9 Product Discovery Precision and Recall (Fig. 11)

[GRAPH PLACEHOLDER]

Grouped bar chart.
X-axis: Product category. Y-axis: Percentage (0–100%).
Metric pairs: Precision & Recall for TalknShop vs. Manual.

Figure 11: Search precision and recall by product category.

7 Implementation Status

Table 2 presents the current implementation status as of March 2026.

Table 2: Implementation Status (March 2026)

Component	Status	Notes
Orchestrator	85%	WebSocket + LangGraph functional
Catalog Service	20%	Structure done; adapters in progress
Seller Crosspost	0%	Architecture designed
Media Service	30%	Basic structure; Bedrock pending
Web App (React)	90%	Chat UI + WebSocket working
iOS App	0%	Planned

The stack comprises Python 3.11+/FastAPI (backend), React/TypeScript (frontend), Docker Compose (local development), and AWS CDK for infrastructure. AWS services include Bedrock (Claude 3), DynamoDB, SQS, S3, and ECS.

8 Limitations and Future Work

The evaluation framework proposed in Section VI has not yet been executed; visualizations represent planned analyses pending user study data. The Catalog Service adapters are at an early stage, and the Seller Crosspost Service is unimplemented. Near-term priorities include completing marketplace adapters, implementing the Crosspost Service with SQS workers, and conducting user studies. Medium-term goals include the iOS app, preference-learning ranking algorithms, and additional marketplace support. Longer-term directions include agentic commerce capabilities for autonomous purchases, voice-first interaction, and visual search for cross-marketplace product matching.

9 Conclusion

This paper presented TalknShop, a microservices-based conversational AI platform unifying multi-marketplace shopping through natural language. By combining LangGraph-based orchestration, AWS Bedrock-powered NLU, marketplace adapter abstractions, and separated synchronous/asynchronous service architectures, TalknShop provides a foundation for a new paradigm in conversational commerce. Industry evidence showing AI-assisted shoppers converting at nearly 4× the rate of unassisted shoppers validates the potential impact. Our proposed evaluation framework with ten visualizations will provide comprehensive empirical evidence once data collection is complete.

Acknowledgment

This work is part of a Master's Project at San Jose State University, Department of Computer Science. The authors thank their project advisor for guidance on system architecture and research methodology.

References

[1] Neuwark Research, "Conversational commerce in 2026: AI is replacing the shopping cart," 2026. [Online]. Available: https://neuwark.com/blog/conversational-commerce-2026

[2] Rep AI, "The future of AI in ecommerce: 40+ statistics on conversational AI agents for 2025," 2025.

[3] Bloomreach, "More than 60% of consumers have used conversational AI for shopping," Press Release, Mar. 2025.

[4] C. Messina, "Conversational commerce," Medium, Jan. 2016.

[5] A. Rese, S. Ganster, and D. Baier, "Conversational commerce: Entering the next stage of AI-powered digital assistants," *Ann. Oper. Res.*, Springer, 2021.

[6] ALM Corp, "ChatGPT shopping research: The complete guide to AI-powered product discovery," 2025.

[7] A. Masood, "Chat-driven commerce: The rise of inline shopping in conversational AI," Medium, Nov. 2025.

[8] LangChain, "LangGraph: Build resilient language agents as graphs," 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[9] S. Newman, *Building Microservices*, 1st ed. O'Reilly Media, 2015.

[10] C. Richardson, *Microservices Patterns*, 1st ed. Manning, 2018.

[11] Grand View Research, "U.S. AI shopping assistant market report, 2033," 2025.

[12] Straits Research, "AI shopping assistant market size, share, trends forecast by 2034," 2025.

[13] J. Lewis and M. Fowler, "Microservices: A definition of this new architectural term," martinfowler.com, Mar. 2014.

[14] Walmart, "Retail Rewired Report 2025," 2025.

[15] McKinsey & Company, "The state of AI in 2025," Global Survey, 2025.